4.2. GIỚI THIỆU VỀ NPM

4.2.1. Khái niệm NPM

NPM viết tắt của **Node Package Manager** là một công cụ tạo và quản lý các thư viện lập trình **Javascript** cho **NodeJS**. Trong cộng đồng Javascript, các lập trình viên chia sẻ hàng trăm nghìn các thư viện với các đoạn code đã thực hiện sẵn một chức năng nào đó. Nó giúp cho các dự án mới tránh phải viết lại các thành phần cơ bản, các thư viện lập trình hay thậm chí cả các framework.

Nếu bạn vẫn chưa tưởng tượng ra lợi ích của việc sử dụng **npm** thì ta sẽ lấy ví dụ cụ thể hơn:

Ví dụ 1: Nếu bạn không sử dụng npm, bạn sẽ phải download toàn bộ các thư viện một cách thủ công, sau đó include vào project của bạn, việc này rất mất thời gian, trong khi với npm, bạn chỉ cần 1 dòng lệnh là xong.

Ví dụ 2: Khi làm 1 dự án, bạn phải chia sẻ code cùng với các cộng sự của mình, nếu không sử dụng **npm** (hoặc bất kì trình quản lí package nào) thì khi commit (chuyển giao) code ta phải commit cả thư viện vào, rất nặng. Khi deploy (triển khai) ta cũng phải copy thư viện lên, rất chậm và tốn thời gian.

4.2.2. Cài đặt NPM

- Tạo thư mục **Chuong4** trong thư mục **D:\THNodejs** đã có.

- Tạo thư mục **Vidu4_1** trong thư mục **Chuong4**.

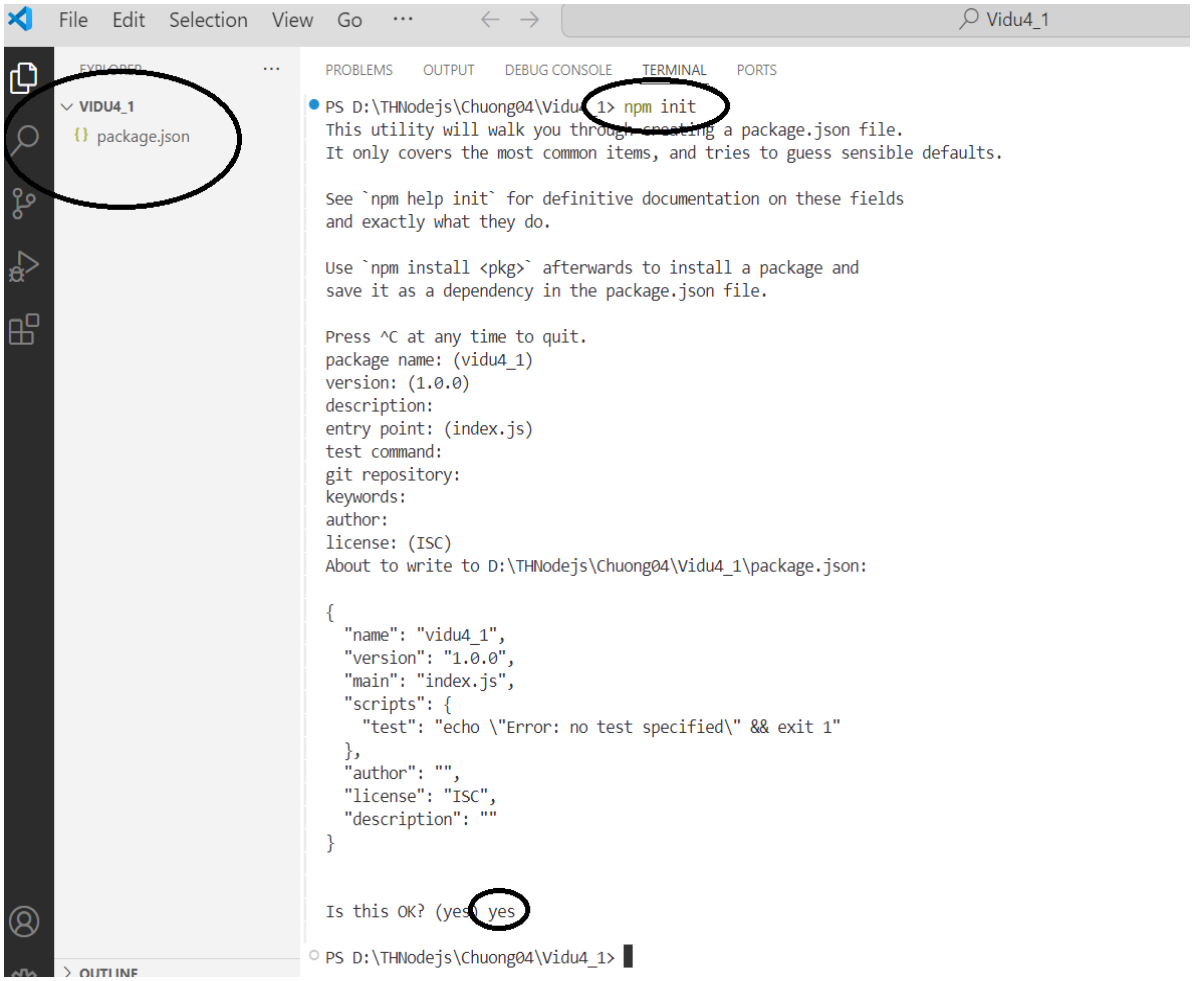
- Mở **VSCode** lên, mở thư mục **Vidu4_1** trong **VSCode**

- Thực hiện câu lệnh khởi tạo npm: **npm init**

Sau khi chạy câu lệnh này, Npm sẽ hỏi chúng ta một vài câu hỏi về project của bạn. ***(Các bạn có thể Enter liên tục để mặc định các giá trị)***

- Đầu tiên là về package name: Đây là tên dự án của bạn
- Version: Đây là phiên bản dự án của bạn, mặc định sẽ là 1.0.0
- Description: Mô tả về dự án của bạn.
- Entry point: Entry point sẽ quy định root của Node, file này sẽ chứa các câu lệnh về server, mặc định sẽ là index.js. Chú ý là khi bạn chạy server, Node sẽ đọc dữ liệu từ file này để khởi tạo. Tên này có thể thay đổi được, nhưng hãy chắc rằng file đó tồn tại ở root của dự án.
- Test command: Đây là dòng lệnh mà sẽ chạy khi bạn gọi npm test Mặc định để trống
- Git repository: Như tên gọi, nó là git repository ứng với dự án của bạn. Mặc định để trống.
- Keywords: Các từ khóa tương ứng với dự án của bạn. Mặc định để trống
- Author: Tên tác giả của dự án. Mặc định để trống
- License: Giấy phép. Mặc định là ISC.

- Khi kết thúc sẽ hỏi 1 câu là Is this OK: bạn có chắc không, yes hoặc no. Các bạn gõ **yes**



- Sau khi kết thúc quá trình khởi tạo, **project** của các bạn sẽ có thêm **1 file** có tên là **package.json**. Khi bạn vào trong đó ban đầu sẽ file hiển thị những thông tin tương tự như dưới đây:

- Các thông tin trong file này ban đầu chính là những thông tin bạn nhập trước đó trong phần **npm init**. Trong tương lai bạn có thể **add** rất nhiều thứ vào **file** này nữa. Và giờ bạn có thể sử dụng **npm** để cài đặt các **package** mà mình muốn **include** vào trong dự án của mình rồi. (Ví dụ khi sử dụng câu lệnh: **npm i nodemon** thì các bạn đã cài package nodemon vào dự án của mình)

- Dùng lệnh: **npm -v** để xem phiên bản npm:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS D:\THNodejs\Chuong04\Vidu4_1> npm -v
10.8.1
PS D:\THNodejs\Chuong04\Vidu4_1>
```

- Dùng lệnh: **npm list** để xem các package đã cài vào npm:

```
PS D:\THNodejs\Chuong04\Vidu4_1> npm list
vidu4_1@1.0.0 D:\THNodejs\Chuong04\Vidu4_1
└── (empty)

PS D:\THNodejs\Chuong04\Vidu4_1>
```

4.2.3. Các loại Package và các kiểu cài đặt Package

Các loại package:

Dựa theo chức năng mà ta chia package ra làm **2** loại, đó là **Simple dependencies** (có thể chỉ gọi nó là **Dependecies**) và **Development dependencies**. Về sự khác nhau giữa 2 loại này ra sao.

Dependencies: là những package bắt buộc phải có trong quá trình chạy sản phẩm, kiểu như một thư viện cung cấp các hàm mà code của bạn cần. Khi cài đặt dependencies, **npm** sẽ tự động cài đặt tất cả các dependencies cần thiết.

Ví dụ như nếu package A phụ thuộc vào package B, package B lại phụ thuộc vào package C thì **npm** sẽ cài đặt đồng thời cả A,B,C.

Đó cũng chính là lí do vì sao khi bạn cài đặt 1 package và vào folder node_modules sẽ thấy cả đống các package khác mà bạn không hề cài đặt.

+ ***Cài đặt Simple dependencies package:***

Cú pháp như sau: **npm install [tên package]**

hoặc đơn giản hơn là: **npm i [tên package]**

Ví dụ: npm i nodemon; npm i express

Còn đối với **Development dependencies**: là những package bắt buộc khi phát triển cũng như phát hành sản phẩm. Kiểu như các trình biên dịch giúp biên dịch đoạn code của bạn về javascript, rồi các framework phục vụ cho việc kiểm thử,…

Khi cài đặt Development Dependencies, **npm** sẽ chỉ cài đặt các dependencies mà cần thiết.

Ví dụ như nếu package A phụ thuộc vào package B, package B lại phụ thuộc dev vào package C thì **npm** sẽ chỉ cài đặt A và B.

Giờ chúng ta sẽ xem chúng ta cài đặt các package như thế nào nhé.

+ Cài đặt Development dependencies package:

Cú pháp như sau: **npm install [tên package] --save-dev**

hoặc đơn giản hơn là: **npm i [tên package] --save-dev**

Ví dụ: npm i nodemon --save-dev; npm i express --save-dev

Các kiểu cài đặt package:

Chúng ta có 2 kiểu cài đặt package: **Local** và **Global**

Ví dụ các kiểu cài đặt ở trên là cài đặt local, tức là các package ở trên được cài đặt và chỉ sử dụng được trong project hiện tại mà nó được cài đặt, với các project khác sẽ không sử dụng được.

Kiểu cài đặt thứ 2 là global, các package sẽ được cài đặt global, đúng như tên gọi của nó, tức là bạn có thể sử dụng nó ở không chỉ project hiện tại mà ở các project khác mà không cần phải cài đặt lại nữa.

Để cài đặt package kiểu **global**, bạn viết câu lệnh giống như của Simple Dependencies thêm flag **-g** hoặc **-global**

Ví dụ: *npm i nodemon -g*